



UNIVERSITÉ DES SCIENCES ET DE LA TECHNOLOGIE HOUARI BOUMEDIENE
FACULTE D'ELECTRONIQUE ET D'INFORMATIQUE
LABORATOIRE DES SYSTÈMES INFORMATIQUES



LES ENTREPOTS DE DONNEES Data Warehouse

Dr. Kamel Boukhalfa

Boukhalk@gmail.com

1

Présentation de l'enseignant

- **Dr. Kamel Boukhalfa**

- ☐ Docteur de l'USTHB et de l'ENSMA France
- ☐ Professeur à l'USTHB

- **Emails**

- ☐ Email de l'USTHB : kboukhalfa@usthb.dz
- ☐ Emails souvent utilisés
 - ☐ boukhalk@gmail.com
 - ☐ Kamelboukhalfa@yahoo.fr

- **Site Web personnel**

- ☐ <http://boukhalfa.iimdo.com>
- ☐ CV
- ☐ Production scientifique
- ☐ Téléchargement de cours



LES ENTREPOTS DE DONNEES - Data Warehouse

Domaines de Recherches

☐ Entrepôts de données

- ☐ Optimisation de la couche physique, Fragmentation Horizontale, Indexation, Tuning

☐ Méta-heuristiques

- ☐ AG, RS, HC, RT, CF, ...etc.
- ☐ Optimisation multi-objectifs

☐ Data mining

- ☐ Motifs fréquents, classification, règles d'association, data mining spatial

☐ Big Data

☐ Cloud Computing



Plan

☐ Les entrepôts de données

☐ Cycle de développement

☐ Architectures

☐ Modélisation

☐ Optimisation des entrepôts de données



Le contexte

- ❑ **Besoin:** prise de décisions stratégiques et tactiques
- ❑ **Pourquoi:** besoin de réactivité par à un monde qui change rapidement
- ❑ **Qui:** les décideurs (non informaticiens)
- ❑ **Comment:** répondre aux demandes d'analyse des données, dégager des informations qualitatives nouvelles



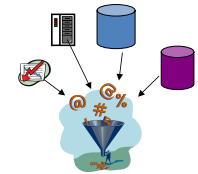
Les données utilisables par les décideurs

❑ Données opérationnelles (de production)

- Bases de données (Oracle, SQL Server)
- Fichiers, ...
- Paye, gestion des RH, gestion des commandes...

❑ Caractéristiques de ces données:

- **Distribuées:** systèmes éparpillés
- **Hétérogènes:** systèmes et structures de données différents
- **Détaillées:** organisation des données selon les processus fonctionnels, données surabondantes pour l'analyse
- **Peu/pas adaptées à l'analyse :** les requêtes lourdes peuvent bloquer le système transactionnel
- **Volatiles:** pas d'historisation systématique



Problématique

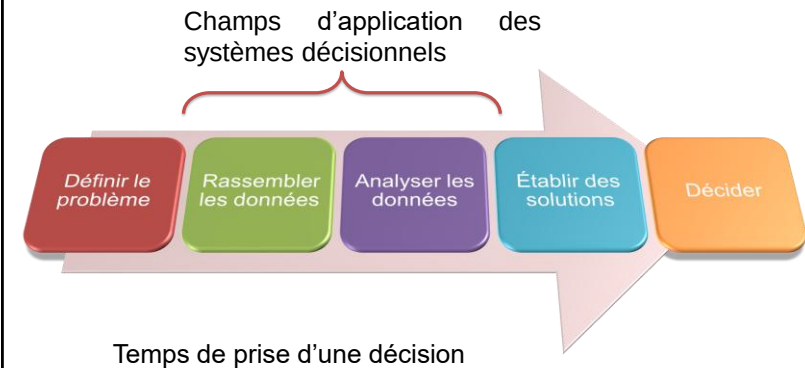
- Comment répondre aux demandes des décideurs?
 - En donnant un accès rapide et simple à l'information stratégique
 - En donnant du sens aux données

Mettre en place un système d'information dédié aux applications décisionnelles

→ **Un entrepôt de données (Data Warehouse)**



Le processus de prise de décision



Introduction

❑ Pourquoi le data warehouse ?

- Améliorer les performances **décisionnelles** de l'entreprise

❑ Comment?

- En répondant aux demandes d'analyse des décideurs

❑ Exemples

- **Clientèle:** **Qui** sont mes clients? **Pourquoi** sont-ils mes clients?, **Comment** les **conserver** ou les faire revenir (préférence d'achat, habitudes, ...)? Ces clients sont-ils vraiment intéressants pour moi?
- **Marketing, actions commerciales:** où placer ce produit dans les rayons? Comment cibler plus précisément le mailing concernant ce produit?



Introduction

Raisons d'être d'un entrepôt de données

- Rassembler les données de l'entreprise dans un **même lieu** sans **surcharger** les BD (systèmes opérationnels)
- Permettre un **accès universel** à diverses sources de données et assurer la **qualité** des données
- **Extraire**, **filtrer**, et **intégrer** les informations pertinentes, à l'avance, pour des requêtes ultérieures
- Dégager des **connaissances** et faire un apprentissage sur l'entreprise, le marché et l'environnement



C'est quoi un entrepôt de données?

❑ Industrie (Inmon 1992)

- Collection de données **orientées sujets**
- Consolidées dans une **base de données unique**
- **Non volatiles** et **historisées variant** dans le **temps**
- organisées pour le support d'un **processus d'aide à la décision**

❑ Recherche (Stanford 1995)

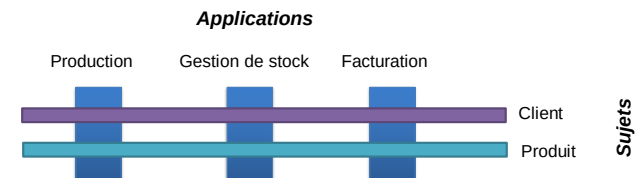
- Dispositif de stockage d'informations **intégrées** de sources **distribuées, autonomes, hétérogènes**



Les 4 caractéristiques des data warehouse

1. Données orientées sujet

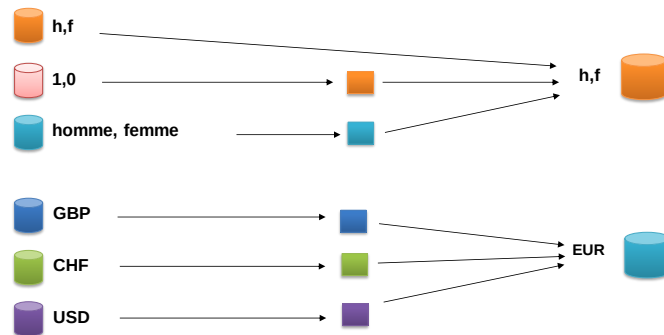
- Regroupe les informations des différents métiers
- Ne tiens pas compte de l'organisation fonctionnelle des données



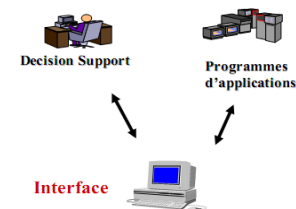
Les 4 caractéristiques des data warehouse

2. Données intégrées

- Normalisation des données
- Définition d'un référentiel unique



Intégration de données



Caractéristiques des sources de données :

- Hétérogènes (schématique, sémantique),
- Autonomes
- Évolutives
- Réparties

Besoins :

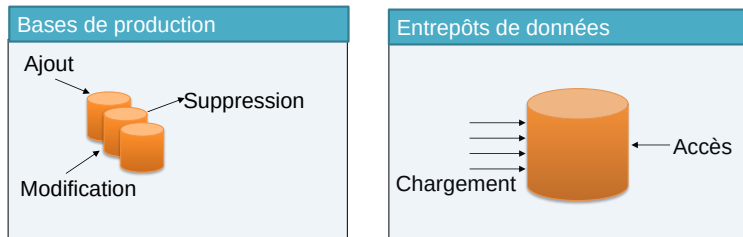
- Intégration données
- Gestion de l'évolution des données



Les 4 caractéristiques des data warehouse

3. Données non volatiles

- Traçabilité des informations et des décisions prises
- Copie des données de production



Les 4 caractéristiques des data warehouse

4. Données datées

- Les données persistent dans le temps
- Mise en place d'un référentiel temps

Base de production

Image de la base en Mai 2005

Répertoire

Nom	Ville
Karim	Alger
Salim	Oran

Image de la base en Juillet 2006

Répertoire

Nom	Ville
Karim	Laghouat
Salim	Oran

Entrepôt de données

Calendrier

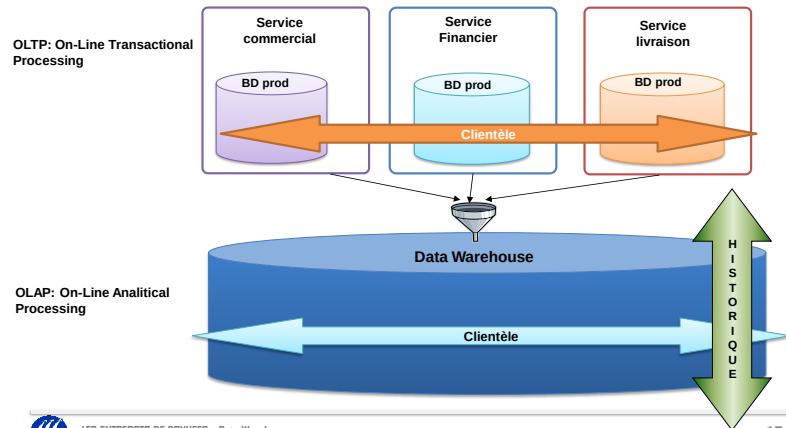
Code	Année	Mois
1	2005	Mai
2	2006	Juillet

Répertoire

Code	Nom	Ville
1	Karim	Alger
1	Salim	Oran
2	Karim	Laghouat



SGBD et DW



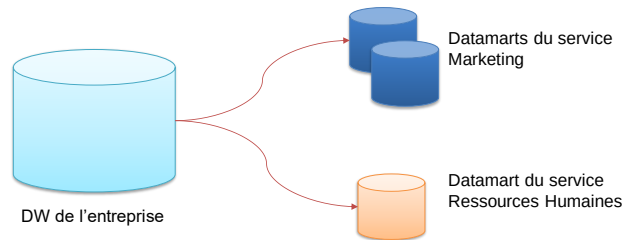
OLTP VS DW

OLTP	DW
Orienté transaction	Orienté analyse
Orienté application	Orienté sujet
Données courantes	Données historisées
Données détaillées	Données agrégées
Données évolutives	Données statiques
Utilisateurs nombreux, administrateurs/opérationnels	Utilisateurs peu nombreux, manager
Temps d'exécution: court	Temps d'exécution: long



Datamart

- ❑ Sous-ensemble d'un entrepôt de données
- ❑ Destiné à répondre aux besoins d'un secteur ou d'une fonction particulière de l'entreprise
- ❑ Point de vue spécifique selon des critères métiers

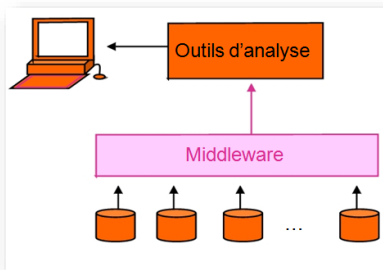


Intérêt des datamart

- ❑ Nouvel environnement structuré et formaté en fonction des besoins d'un métier ou d'un usage particulier
- ❑ Moins de données que DW
 - ❑ Plus facile à comprendre, à manipuler
 - ❑ Amélioration des temps de réponse
- ❑ Utilisateurs plus ciblés

Architecture d'un entrepôt de données

Approche virtuelle (ou le non-entrepôt)



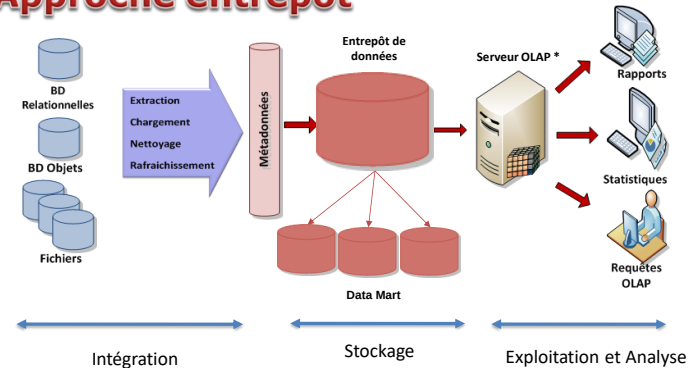
Inconvénients

- ☐ pas de réelle intégration des données
- ☐ différentes vues non-réconciliées
- ☐ pas de vues dans le temps
- ☐ les requêtes peuvent facilement bloquer les transactions en cours



Architecture d'un entrepôt de données

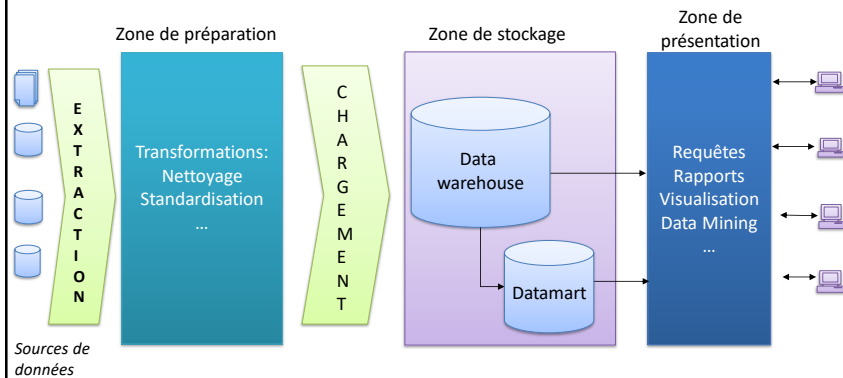
Approche entrepôt



* OLAP : Online Analytical Processing



Architecture générale



Les flux de données

- **Flux entrant**
 - Extraction: multi-source, hétérogène
 - Transformation: filtrer, trier, homogénéiser, nettoyer
 - Chargement: insertion des données dans l'entrepôt
- **Flux sortant**
 - Mise à disposition des données pour les utilisateurs finaux



Les différentes zones de l'architecture

- **Zone de préparation (Staging area)**
 - Zone temporaire de stockage des données extraites
 - Réalisation des transformations avant l'insertion dans le DW:
 - Nettoyage
 - Normalisation...
 - Données souvent détruites après chargement dans le DW



Les différentes zones de l'architecture

- **Zone de stockage (DW, DM)**
 - On y transfère les données nettoyées
 - Stockage permanent des données
- **Zone de présentation**
 - Donne accès aux données contenues dans le DW
 - Peut contenir des outils d'analyse programmés:
 - Rapports
 - Requêtes...

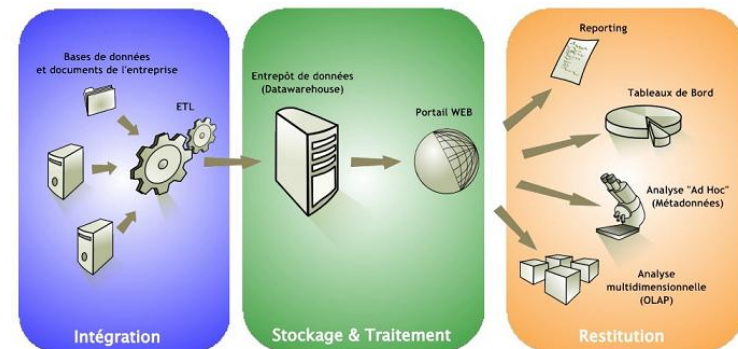


Exemple Suite Pentaho

- **Pentaho est une plate-forme décisionnelle open source complète** possédant les caractéristiques suivantes
- Une couverture globale des fonctionnalités de la Business Intelligence
 - ETL (intégration de données),
 - reporting,
 - tableaux de bord (« Dashboards »),
 - analyse *ad hoc* (requêtes à la demande),
 - analyse multidimensionnelle (OLAP)



Exemple Pentaho



Pentaho - Utilisateurs

Pentaho permet d'adresser deux typologies d'utilisateurs

- ❑ Les « one-clic users », utilisateurs de base, consommateurs d'indicateurs prédéfinis,
- ❑ Les utilisateurs avancés, qui ont besoin d'outils d'analyse et d'exploration avancés ;



Reporting :

- Rapports & états préformatés (pdf, excel, html)
- Filtrage possible des données ("row level")

Tableaux de bords :

- Présentation graphique et synthétique d'indicateurs
- Permettent l'obtention intuitive de rapports ("drill-down")

Analyse Ad hoc :

- Permet de créer des rapports avancés (choix des colonnes, tableaux croisés) à partir de vues métiers ("business views"). Nécessite la mise en place d'une couche sémantique d'abstraction (métadonnées)

Analyse multidimensionnelle (OLAP)

- Permet la manipulation de données selon plusieurs axes d'analyse. Nécessite une modélisation spécifique dans le SGBDR (ROLAP)



Pentaho - Architecture

Une architecture Web 2.0 qui se

compose de :

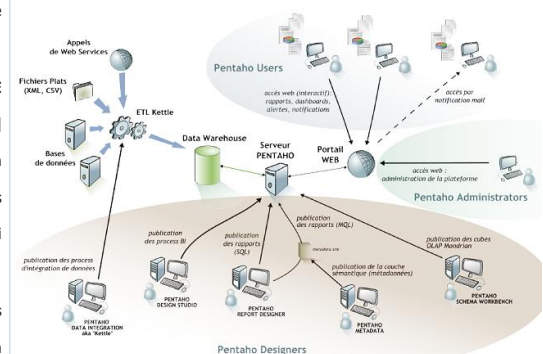
❑ Un serveur Web J2EE

[Pentaho User Console » (PUC)]

permettant de mettre à disposition l'ensemble des ressources décisionnelles et ceci au travers d'URL Web.

❑ Plusieurs clients riches

permettant la conception et la publication des ressources.



Pentaho - Architecture

- ❑ **Une plate-forme d'administration** (*Pentaho Administration Console*) pour la gestion des droits d'accès, la planification d'événements, la gestion centralisée des sources de données;
- ❑ **La plate-forme est orientée « processus »** : au travers de « séquences d'actions » on peut ainsi modéliser avec Pentaho des workflows BI avancés ;
- ❑ **Facilement utilisable** : pas besoin de connaître JAVA pour travailler avec Pentaho (seule la maîtrise du langage SQL est nécessaire, ainsi que des connaissances de base en XML, HTML et JavaScript.)
- ❑ **Une communauté importante** : plugins Kettle, Pentaho Analysis Tool, Pentaho Community Dashboard Framework, etc. ;
- ❑ Pentaho est une suite décisionnelle open source. **La version libre de Pentaho permet d'installer une plate-forme décisionnelle complète**



Exploitation de l'entrepôt

❑ Business Intelligence:

- Possibilité de **visualiser** et d'**exploiter** une masse **importante** de données **complexes**

❑ Trois principaux outils:

- **OLAP** : On-Line Analytical Processing
- **Data mining/IA**: fouille de données, Intelligence artificielle
- Formulation de **requêtes** et **visualisation** des résultats



Architecture d'un entrepôt de données

☐ Souvent une architecture **trois-tiers**

- **Serveur d'entrepôt** ("Warehouse Database Server")
 - Très souvent un système relationnel (ex. Oracle)
- **Serveur OLAP** ("OLAP Server") de type ROLAP, MOLAP, ou HOLAP
- **Clients**
 - Outils de requêtes et de production de rapports
 - Outils d'analyse et de prospection de données



Domaines d'applications

☐ Banque, Assurance

- Détermination des profils client (prêt, ...)

☐ Commerce

- Ciblage de clientèle
- Compagnies de grande production
- Aménagement des rayons (2 produits en corrélation)

☐ Compagnies téléphoniques

☐ Santé



Base de données vs. Entrepôt de donnée

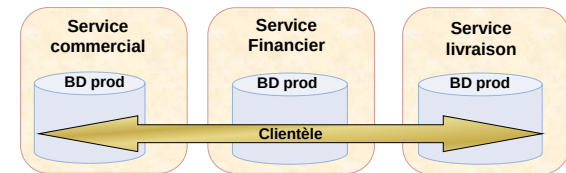
Pourquoi dissocier une BD d'un ED?

- ❑ Les objectifs de **performances** dans les BD ne sont pas les mêmes que ceux dans les EDs :
 - BD : requêtes **simples** (OLTP), méthodes d'accès et indexation
 - ED : requêtes OLAP souvent **complexes!!!**
- ❑ La nécessité de **combiner** des données provenant de diverses sources, d'effectuer des **agrégations** dans un ED et d'offrir des **vues multidimensionnelles**
- ❑ Les données d'un ED sont souvent non **volatiles** et ont donc une plus longue durée de vie que celles d'une BD

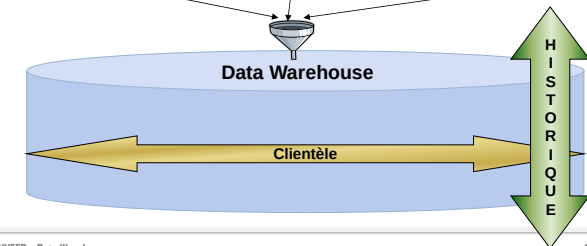


SGBD et DW

OLTP: On-Line
Transactional
Processing



OLAP: On-Line
Analytical
Processing

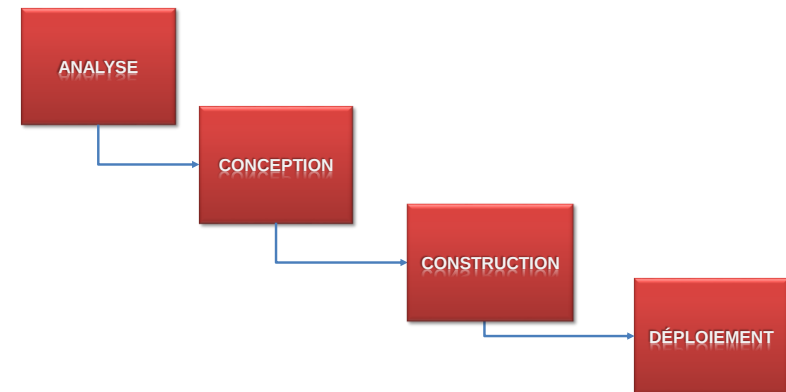


OLTP VS DW

OLTP	DW
Orienté transaction	Orienté analyse
Orienté application	Orienté sujet
Données courantes	Données historisées
Données détaillées	Données agrégées
Données évolutives	Données statiques
Utilisateurs nombreux, administrateurs/opérationnels	Utilisateurs peu nombreux, manager
Temps d'exécution: court	Temps d'exécution: long



Cycle de vie de l'entrepôt de données



Cycle de vie

❑ Spécification des besoins

- Rassembler clairement et fidèlement les besoins des utilisateurs (**décideurs**)
- Clarifier les objectifs spécifiques
 - Comportement de la clientèle, analyse de tendances de prévisions, etc.
- Énumérer les **dimensions**
- Définir l'**architecture du système** (modèle de données), l'**usage final** (rapports, requêtes, **outils d'analyse** et **visualisation**)



Cycle de vie

❑ Analyse

- Développer le schéma de l'entrepôt
- Définir les processus nécessaires à la mise en place de l'entrepôt (extraction de données à partir des sources, transformations)

❑ Conception (3 niveaux)

- **Conceptuel** : mise au point du schéma, définition des méta-données
- **Logique** : adapté aux particularités du serveur de l'entrepôt (ROLAP, MOLAP, etc.)
- **Physique**: choix d'index, vues matérialisées, fragmentation



Cycle de vie

☐ Construction

- Développer des programmes d'extraction, d'épuration et de transformation de données

☐ Déploiement

- Fournir une installation initiale incluant la connexion aux données sources, la synchronisation et la réplication de données
- Permettre des extensions futures
- Offrir la formation pour les groupes d'intervenants
- Offrir les divers mécanismes d'administration de l'ED (reprise, sécurité, performances)
- Offrir les outils nécessaires à l'exploitation des données et à la consultation des méta-données



Outils requis

☐ Des outils d'accès aux BD

☐ Des outils d'intégration des données des BD vers l'ED

☐ Des outils de nettoyage de données

☐ Le gestionnaire de l'entrepôt de données

☐ Des outils de réplication



Modélisation classique (OLTP)

❑ Le modèle relationnel

- Table, attributs, tuples, vues, ...
- Normalisation (redondance)
- Requêtes simples (sélection, projection, jointure, ...)

❑ Le critère **temps**

- Représentation du passé
 - Un fardeau pour les systèmes OLTP



Requêtes décisionnelles plus complexes !!

❑ Exemples

- ❑ Combien de clients âgés entre 20 et 30 ans et résidant à Alger ont-ils acheté une caméra vidéo au cours des 5 dernières années ?
- ❑ Quelle est la répartition des ventes par produit, ville et par mois au cours de la présente année?
- ❑ Quelles sont les composantes des machines de production ayant eu le plus grand nombre d'incidents imprévisibles au cours de la période 1992-97 ?

➡ Critère **temps** est la base de l'analyse décisionnelle



Récapitulatif

	Base de données	Entrepôt de données
Opération	▪ Gestion courante	▪ Support à la décision
Modèle	▪ Entité association	▪ Etoile, flocon de neige
Normalisation	▪ Plus fréquente	▪ Rare
Données	▪ Actuelle, brutes	▪ Historiques, agrégées
Mise à jour	▪ immédiate	▪ Plus différée
Perception	▪ Bidimensionnel	▪ Multidimensionnelle
Opérations	▪ Lecture/écriture	▪ Lecture et rafraichissement
Taille	▪ Des giga-octets	▪ Vers des téra, péta-octets



OLAP

- ❑ Traitement analytique interactif (Codd) typique dans les systèmes informationnels
- ❑ Catégorie de traitements dédiés à l'aide à la décision
- ❑ Analyses diverses (multidimensionnelles)
- ❑ Information : surtout dérivée et sommaire
- ❑ Aide à la prise de décision



Modélisation Multidimensionnelle

❑ Dimension:

- Présente le point de vue selon lequel on veut voir le données décrites par un ensemble d'attributs· Axe de l'analyse
 - **Exemple:** Commandes, achats, réclamations, produits, clients,...

❑ Mesures/faits

- Fonction numérique qui peut être évaluée en tout point du data cube en agrégeant les données correspondant à ce point
- Mesure d'activité (critère d'analyse)
 - **Exemple:** Chiffre d'affaire, nombre de ventes, gain



Hyper cube OLAP

❑ Objectifs

- Obtenir des informations déjà agrégées selon les besoins des utilisateurs
- Représentation de l'information dans un hyper cube à N dimensions

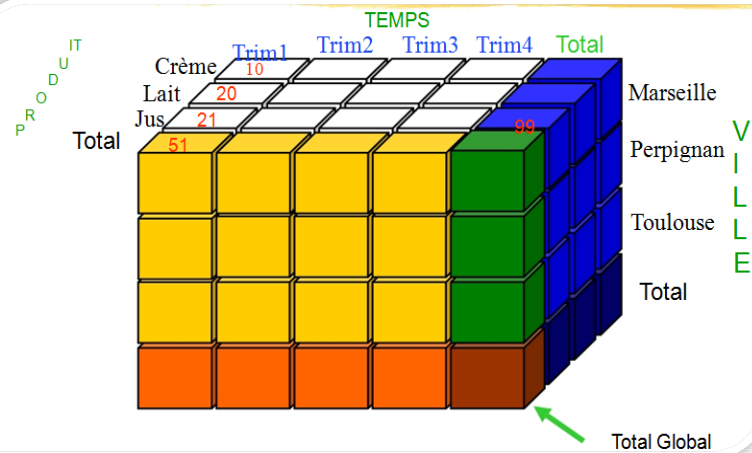


❑ Opérations OLAP

- Fonctionnalités qui servent à faciliter l'analyse multidimensionnelles: **opérations réalisables sur l'hyper cube**



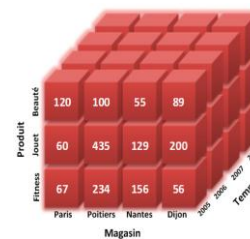
Exemple d'un cube de données



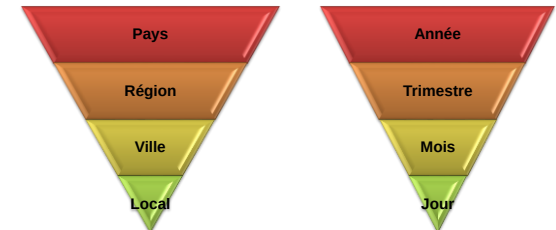
Vue multidimensionnelle des données

❑ Le volume des ventes (**mesure**) en fonction de : Produit, Temp et Localisation (Dimension)

Dimensions: Produit, Région, Temps



Hiérarchie des dimensions



Comment stocker le cube de données

❑ ROLAP: Relational On-Line Analytical Processing

- Les données sont stockées comme des tables relationnelles: une table de faits et des tables de dimension

❑ MOLAP: Multidimensional On-Line Analytical Processing

- le cube de données est stocké sous forme d'un tableau multi-dimensionnel.



Modèle ROLAP

❑ Exploiter l'expérience des modèles relationnels (un grand succès!!)

❑ Il faut des modèles bien adaptés aux ED!

- Schéma en étoile (star schema)
- Schéma en flocon de neige (snowflake schema)
- Schéma en constellation

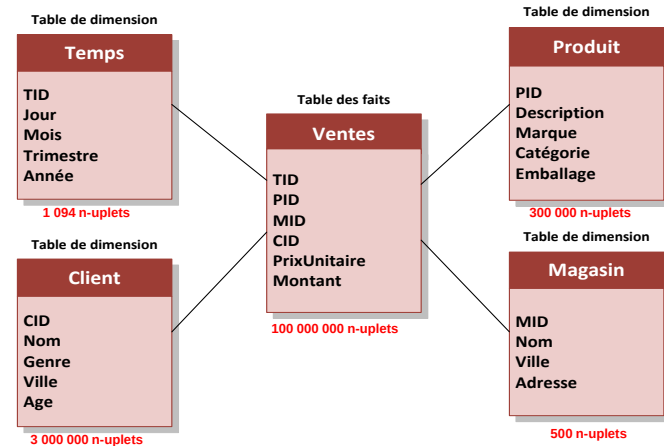


Modèle en étoile

- ❑ Autant de tables de dimension qu'il existe de dimensions
 - **Exemple**
 - Temps, Produit, Client
- ❑ Une table de faits contenant la clé de chaque dimension et des mesures
 - **Exemple**
 - Montant en dollars, nombre d'unités vendues



Schéma en étoile



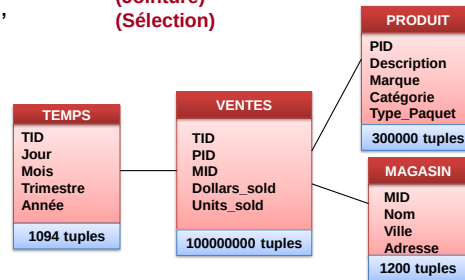
Requête type

Requête de Jointure en étoile :

```
SELECT P.Marque, sum(dollars_sold),  
sum(units_sold)  
FROM Ventes V, PRODUIT P, TEMPS T  
WHERE V.PID = P.PID (Jointure)  
AND V.TID = T.TID (Jointure)  
AND T.Trimestre = 'T1' (Sélection)  
GROUP BY P.Marque  
ORDER BY P.Marque
```

Requêtes de jointure en étoile

- Plusieurs jointures
- Suivies par des sélection



Avantages & Inconvénients

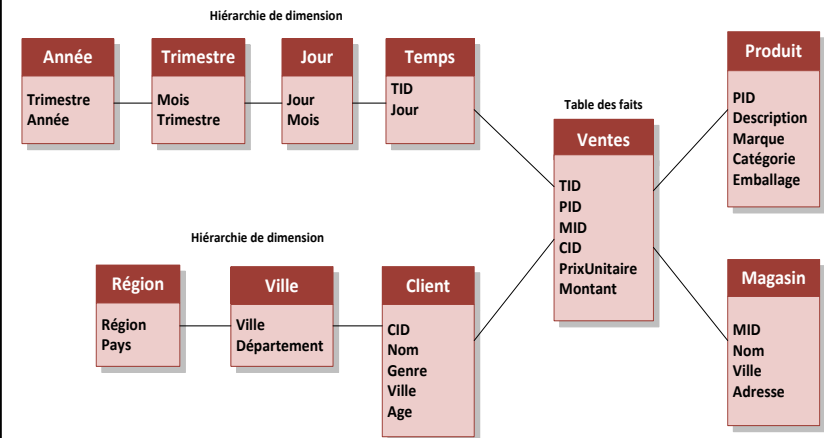
- + Simple
- + Le plus utilisé !!!
- Possibilité de redondance car les tables de dimension ne sont pas nécessairement normalisées
- Taille de dimensions plus grosse

Modèle en flocon de neige

- ❑ Variante du modèle en étoile
- ❑ Les tables de dimension sont normalisées
- ❑ Réduction de la redondance mais exécution parfois plus lente des requêtes (jointure de tables)
- ❑ Modèle adopté par **Oracle**!!



Exemple d'un modèle en flocon de neige



Définition d'un schéma en étoile avec DMDL

define cube ventes_star [Temps, Produit, Client]

Montant_vente = sum(somme),

moyenne_vente = avg(somme),

unités_vendues = count(*)

Mesures

Dimensions

define dimension Temps **as** (TID, Année, mois, jour)

define dimension Produit **as** (PID, nom_item, marque, taille, poids)

define dimension Client **as** (Cid, Nom, Sexe, Age, Ville)



Définition d'un schéma snowflake avec DMQL

define cube ventes_snowflake [temps, item, branche, lieu]

Montant_vente = sum(somme),

moyenne_vente = avg(somme),

unités_vendues = count(*)

Mesures

Dimensions

define dimension item **as** (Id_item, nom_item, marque, type,

fournisseur(Id_fournisseur, type_fournisseur))

define dimension branche **as** (Id_branche, nom_branche, type_branche)

define dimension temps **as** (Id_temps, jour, jour_semaine, mois, trimestre, année)

define dimension lieu **as** (Id_lieu, rue, ville(Id_ville, département, pays))

Hiérarchies



MOLAP: Représentation Tableaux

Produit	Temps	Trimestre 1			Trimestre 2			Trimestre 3			Trimestre 4			Total		
	Ville	P	N	Total	P	N	Total	P	N	Total	P	N	Total	P	N	Total
TV LCD		12	34	46	22	36	58	24	37	61	33	55	88	91	162	253
Lecteur DVD		29	66	95	44	50	94	56	55	111	44	39	83	173	210	383
Caméoscope		55	34	89	69	27	96	31	26	57	68	70	138	223	157	380
Total		96	134	230	135	113	248	111	118	229	145	114	309	487	529	1016

Répartition des ventes par produit, temps et ville

P : Poitiers, N : Nantes

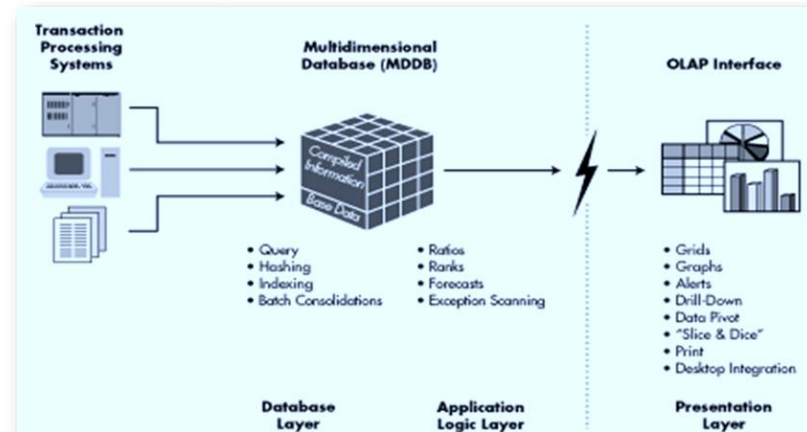
ROLAP vs. MOLAP

	Avantages	Inconvénients
ROLAP	- Standard bien établi - Efficace pour le transactionnel - Capacité d'expansion (téra-octet)	- Absence de vue conceptuelle - SQL peut être inadéquat pour la formation de tableaux croisés
MOLAP	Implémentation souvent plus performante que ROLAP	- Inadéquat pour le transactionnel - Capacité d'expansion limitée (dizaine de giga-octet) - Absence de standard

Serveurs ROLAP [Microstrategy]



Serveurs MOLAP



Hybrid OLAP

- ☐ HOLAP combine ROLAP et MOLAP
- ☐ Les données sont partitionnées en deux sous-ensembles
 - ☐ Données non fréquentes sont stockées comme tables relationnelles
 - ☐ Données fréquentes sont stockées comme tableaux multidimensionnels
- ☐ Données **brutes** dans ROLAP
- ☐ Données **agrégées** dans MOLAP
- ☐ Cette séparation est transparente pour l'utilisateur.

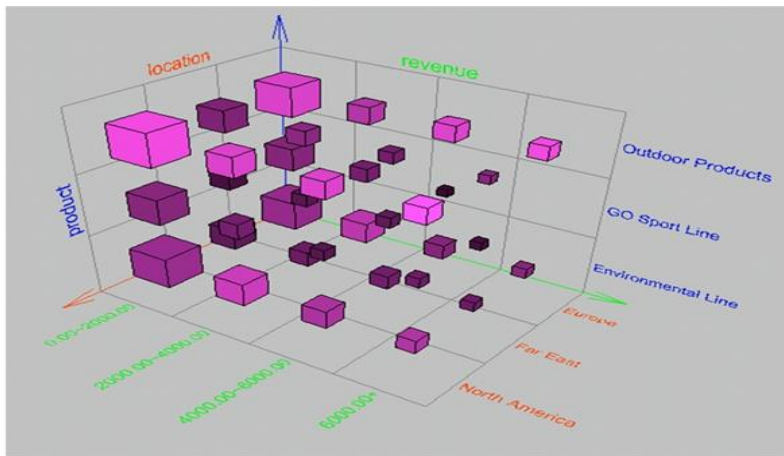


Opérations de base OLAP

- ☐ Group By
- ☐ Roll-up (ou drill-up)
- ☐ Drill-down
- ☐ Slice & dice
- ☐ Pivot
- ☐ Opérations OLTP classiques



Navigation dans un cube



Opérations multidimensionnelles

- ❑ Agrégations (consolidation) de données (pré- calculées)
 - ❑ Réduction de temps de calcul
- ❑ Définition de hiérarchies dans les dimensions («granularités»)
- ❑ Exemples
 - ❑ Temps: jours/mois/trimestres/années
 - ❑ Géographie: ville/région/pays



ROLL-UP

- ❑ Agrégation des mesures en allant d'un niveau particulier de la hiérarchie vers un niveau plus général
- ❑ Exemples
 - ❑ Ayant le total des ventes par ville, ROLL-UP permet de donner le total des ventes par âys

Syntaxe:

```
SELECT ...  
GROUP BY  
ROLLUP(Grouping-column-reference_list)
```

Drill-Down: opération inverse de roll-up



Slice & dice

- ❑ **Slice et dice**
 - ❑ **projection et sélection du modèle relationnel**
- ❑ **Pivot (rotate)**
 - ❑ **Réoriente le cube pour visualisation**



Optimisation des Entrepôts de Données

80

Optimisation des requêtes

❑ Caractéristiques des entrepôts

- Grand volume de données (TB)
- Requêtes décisionnelles complexes
- Peu de mises à jour (ajout seulement!)

❑ Exigence des décideurs

- Temps de réponse raisonnable
- Respect des délais

Objectif Exécution rapide des requêtes $\forall$ la complexité des requêtes ou des données

➡ **Nécessité de techniques d'optimisation**

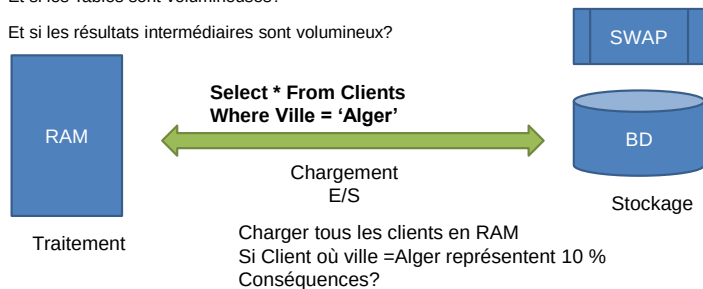


LES ENTREPOTS DE DONNÉES - Data Warehouse

81

Introduction

- ❑ Ce que veut dire optimisation?
- ❑ Qu'est-ce qu'on optimise?
- ❑ Pourquoi il y a un espoir d'optimisation des requêtes?
- ❑ Quelles sont les sources du temps supplémentaire?
- ❑ Et si les Tables sont volumineuses?
- ❑ Et si les résultats intermédiaires sont volumineux?



Évaluation des Requêtes: Survol

- ❑ Évaluation d'une requête SQL:
 - Analysée syntaxiquement, ensuite transformée en plan d'évaluation.
 - **Plan d'évaluation:** Arbre d'opérations de l'algèbre relationnelle avec un choix d'algorithme pour chaque opération.
- ❑ Deux problématiques importantes dans l'optimisation:
 - Énumération des plans alternatifs pour une requête
 - Estimation des coûts de ces plans et choix de celui estimé être le moins cher
- ❑ On doit se montrer pragmatique:
 - **Idéalement:** Trouver le meilleur plan.
 - **Pratiquement:** Éviter les pires plans!

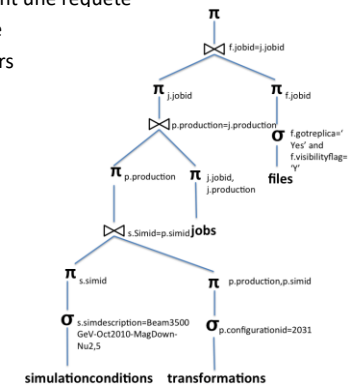
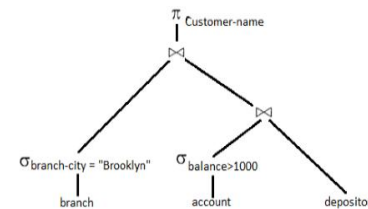
Arbre Algébrique

- ❑ Algèbre relationnelle : modèle formel permettant d'exprimer et de calculer les requêtes sur les relations.
 - ❑ Constituée d'un ensemble d'opérateurs algébriques : la sélection, la jointure, la projection, les opérateurs ensemblistes.
 - ❑ Le résultat de l'exécution d'un opérateur est toujours une relation, ce qui permet la composition.
 - ❑ Utilisée également pour l'optimisation de requêtes.
 - ❑ Une requête algébrique peut se présenter sous forme d'un arbre algébrique.
- Arbre algébrique
- ❑ visualisation d'une requête sous une forme graphique d'arbre plus facile à lire qu'une forme linéaire



Arbre Algébrique

- Un arbre algébrique est un arbre représentant une requête
- ❑ Les nœuds feuilles sont les relations de base
- ❑ Les nœuds intermédiaires sont les opérateurs
- ❑ Le nœud racine est le résultat
- ❑ L'arc est un flux de données



Statistiques et Catalogues

- L'évaluateur a besoin d'informations sur les relations ainsi que les indexes impliquées. Les *Catalogues* contiennent:
 - pour chaque relation: # tuples (NTuples) et # pages (NPages)
 - pour chaque index: # valeurs de clés distinctes (NKeys) et # pages (NPages)
 - pour chaque index à arbre: Hauteur de l'Index (Height), plus petites / plus grande valeurs de clé (Low/High)
- Catalogues rafraîchis périodiquement.
 - Automatiquement, Manuellement
- D'autres types d'infos détaillées (p.ex., histogrammes des valeurs dans certains champs) sont aussi stockées.
 - **Histogramme**: structure donnant une approximation de la distribution des valeurs des données



Chemins d'Accès et Correspondance d'Index

- ❖ Un chemin d'accès est une méthode pour puiser les tuples:
 - Utilisation du Scannage du fichier, ou d'un index qui correspond à ("match") une sélection (WHERE) de la requête.
- ❖ Un index à arbre correspond à une conjonction de termes qui mentionnent seulement des attributs d'un préfixe de la clé de recherche.
 - Index b-arbre sur a correspond à la sélection $a=5$, ou $a>5$
- ❖ Un index à hachage correspond à une conjonction de termes qui a un terme de la forme attribut = valeur pour chaque attribut de la clé de recherche de l'index.
- ❖ Index Bitmap
- ❖ Index de jointure Bitmap



Modes d'optimisation

❑ Expressions Algébriques

- Transformation des requêtes pour obtenir la meilleure séquence d'opérations
- **RBO : Rules Based Optimisation**

❑ Opérations Algébriques

- Coût des différentes stratégies possibles en fonction des caractéristiques des fichiers sur lesquels sont implantées les relations
- **CBO : Cost Based Optimisation**
- Généralement les SGBDs commerciaux combinent les deux stratégies
 - Oracle utilise actuellement CBO seulement



Principes d'optimisation des expressions algébriques

1. Exécuter les sélections aussitôt que possible
2. Réduire la taille des relations à manipuler
 - **Combiner les sélections avec un produit cartésien pour aboutir à une jointure**
3. Combiner des séquences d'opérations unaires (σ, Π)
4. Mémoriser les sous-expression commune



Exécuter les sélections aussitôt que possible

□ Appliquer ces règles (remplacer membre gauche par droit)

1. $\sigma_F (E1 \cup E2) = \sigma_F(E1) \cup \sigma_F(E2)$

2. $\sigma_F (E1 - E2) = \sigma_F(E1) - \sigma_F(E2)$

3. $\sigma_F (E1 \text{ OP } E2) = \sigma_F (E1) \text{ OP } E2$ si F ne porte que sur les attributs de E

4. $\sigma_F (E1 \text{ OP } E2) = \sigma_{F1} (E1) \text{ OP } \sigma_{F2} (E2)$ si $F = F1 \wedge F2$ et F_i ne porte que sur les attributs de E_i

5. $\sigma_F (E1 \text{ OP } E2) = \sigma_{F2} (\sigma_{F1} (E1) \text{ OP } E2)$ si $F1$ porte sur les attributs de $E1$ et $F2$ sur ceux de $E1$ et $E2$

- OP = * ou $\bowtie$



Un algorithme d'optimisation

1. Séparer chaque sélection $\sigma_{F1 \wedge \dots \wedge Fn}(E)$ en une cascade $\sigma_{F1} (\sigma_{F1} \dots (\sigma_{Fn}(E)))$
2. Descendre chaque sélection le plus bas possible dans l'arbre algébrique
3. Descendre chaque projection aussi bas possible dans l'arbre algébrique
4. Combiner des cascades de sélection et de projection dans une sélection seule, une projection seule, ou une sélection suivie par une projection
5. Regrouper les nœuds intérieurs de l'arbre autour des opérateurs binaires ($\bowtie$, *, -, $\cup$). Chaque opérateur binaire crée un groupe
6. Produire un programme comportant une étape pour chaque groupe, à évaluer dans n'importe quel ordre mais tant qu'aucun groupe ne soit évalué avant ses groupes descendants.



CBO - Notions utiles

- ❑ Les tables relationnelles sont stockées physiquement dans des fichiers sur disque
- ❑ Lorsqu'on veut accéder aux données, on doit transférer le contenu pertinent des fichiers dans la mémoire
 - Fichier = séquence de tuples
 - Bloc (page) = unité de transfert de données entre disque et mémoire
 - Facteur de blocage = nombre de tuples d'une relation qui «tiennent» dans un bloc
 - Coût de lecture (ou écriture) d'une relation = nombre de blocs à lire du disque vers la mémoire (ou à écrire de la mémoire vers le disque)



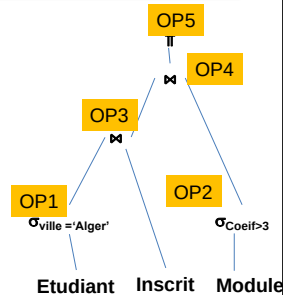
Statistiques au sujet des tables

Statistique	Signification
$ T $	Nombre de lignes de la table T
TailleLigne	La taille d'une ligne de la table T
FBT	Facteur de blocage moyen de T
NDIST'(T _a)	Nombre de valeurs distinctes de la colonne a pour la table T
MinT(colonne)	Valeur minimum de la colonne de T
MaxT(colonne)	Valeur maximum de la colonne de T
$ T $	Nombre de pages de la table T



Coût d'une requête

- ❑ Soit R une requête composée de n opérations algébriques (Opi)
- ❑ Coût (R) = $\sum \text{Coût}(\text{Opi})$
- ❑ Coût (Opi) = Nombre de pages transférées de la mémoire secondaire vers la RAM ou de la RAM vers la mémoire SWAP
- ❑ Pour calculer le Coût(Opi+1)
 - Il faut calculer le nombre de tuples du résultat intermédiaire de Opi ($|\text{Opi}|$)
 - En utilisant le facteur de sélectivité de Opi : $|\text{Opi}| = \text{FS} * |\text{Opi-1}|$
 - Le transformer en nombre de pages en utilisant les tailles des tuples (T_Tuples) et de la page système $|\text{Opi}| = T_tuples * |\text{Opi}| / PS$
 - **Taille tuple** = $\sum \text{taille attributs du tuple}$
 - **Une autre Formule** $|\text{Opi}| = |\text{Opi}| / \# \text{tuples par page}$
- ❑ Si RAM non suffisante : Ajouter les pages transférées en mémoire SWAP



- $T_Tuple(\sigma(T)) = T_Tuple(T)$
- $T_Tuple(T1 \bowtie T2) = T_Tuple(T1) + T_Tuple(T2)$
- $T_Tuple(\pi_{A1, A2, \dots}(T)) = \sum Taille(Ai)$



Facteur de sélectivité (FS)

- ❑ Désigne le pourcentage de lignes sélectionnées sur le nombre total de tuples
 - Entre 0 et 1
 - Opération trop sélective : **SF tend vers zéro**
 - Opération non sélective : **SF tend vers 1**
- ❑ Exemple :
 - Si étudiants habitant Alger représentent **20 %**
 - Le facteur de sélectivité du prédicat **Ville='Alger'** est de **0,2%**
 - Si la table Etudiant contient **100 000 étudiants**, la requête `Select * from Etudiant where Ville = Alger`
 - Retournera **20 000 tuples**



Facteur de sélectivité

□ $||(\sigma(R))|| = SF * ||R||$

- $SF(A = \text{valeur}) = 1 / NDIST(A)$
- $SF(A > \text{valeur}) = (\max(A) - \text{valeur}) / (\max(A) - \min(A))$
- $SF(A < \text{valeur}) = (\text{valeur} - \min(A)) / (\max(A) - \min(A))$
- $SF(A \text{ IN liste valeurs}) = (1/NDIST(A)) * CARD(\text{liste valeurs})$
- $SF(P \text{ et } Q) = SF(P) * SF(Q)$
- $SF(P \text{ ou } Q) = SF(P) + SF(Q) - SF(P) * SF(Q)$
- $SF(\text{not } P) = 1 - SF(P)$

- Le coût dépend de l'algorithme (index, hachage ou balayage).



Facteur de sélectivité

- La taille d'une jointure est estimée par la formule suivante:

– $||R1 \bowtie R2|| = Sel * ||R1|| * ||R2||$

- sel: la sélectivité de la jointure

- Produit cartésien : Sel=1

- Jointure sur clé étrangère sur R1 : $||R1 \infty R2|| = ||R1||$

- Une estimation de Sel si jointure quelconque

– $SF(R1 \bowtie_b R2) = 1 / \text{Max}(NDIST(a), NDIST(b))$



La sélection

❑ Sélection sans index

- `SELECT NSS, Nom FROM Ouvrier WHERE Salaire > 15000`

❑ Plan d'exécution

- Pour chaque tuple t de ouvrier faire -- accès séquentiel
- si $t.salaire > 15000$ alors **réponse** = **réponse** $\cup$ NSS, Nom (Ouvrier)

❑ Coût = $|T| = |\text{Ouvrier}|$

❑ Sélection avec Index

- Charger l'index : Coût = $|\text{Index}|$
- Parcourir l'index en RAM, chercher les tuples à sélectionner: Coût = 0
- Charger les tuples sélectionnés : Coût = $FS * |T| = FS(\text{Salaire} > 15000) * |\text{Ouvrier}|$
- Total = $|\text{Index}| + FS * |T|$

En cas d'absence de statistiques, les estimateurs considèrent une distribution uniforme



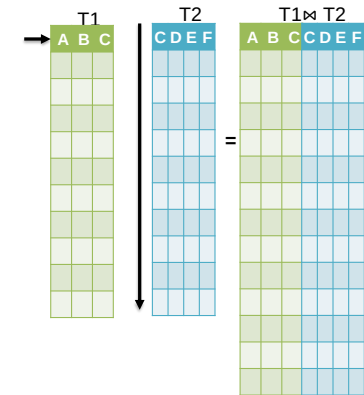
Algorithmes de jointure

❑ Jointure sans index

- Jointure par boucles imbriquées (nested loop)
- Jointure par tri-fusion (sort-join)
- Jointure par hachage (hash-join)

❑ Jointure avec index

- Jointure avec boucles indexées



Jointure par boucles imbriquées (JBI)

- ❑ $R \bowtie S$
- ❑ Principe:
 - La première table est lue séquentiellement et de préférence stockée entièrement en mémoire
 - Pour chaque tuple de R, il y a une comparaison avec les tuples de S

```

POUR chaque ligne IR de R
  POUR chaque ligne IS de S
    Si 0 sur IR et IS est satisfait
      Produire la ligne concaténée à partir de IR et IS
    FINSI
  FINPOUR
FINPOUR
    
```

+ Algorithme simple
 Complexité: $(|R| + (|R| * |S|))$
 Si la table extérieure tient en mémoire: une seule lecture des deux tables suffit.

Jointure par tri fusion

- ❑ Plus efficace que les boucles imbriquées pour les grosses tables
- ❑ Principe
 - Trier les deux tables sur les colonnes de jointure
 - Effectuer la fusion
 - Complexité: Le tri qui coûte cher
- ❑ Très efficace quand une des deux tables est petite (1, 2, 3 fois la taille de la mémoire)
- ❑ Algorithme en option dans Oracle

Coût =

- $|T1| + |T2|$ si T1, T2 déjà triées
- $2 * |T1| * \log_b (|T1| + |T2| + |T1|)$ Si T2 triée
- $2 * |T2| * \log_b (|T2| + |T2| + |T1|)$ Si T1 triée
- $2 * |T1| * \log_b (|T1| + 2 * |T2| * \log_b (|T2| + |T2| + |T1|))$ si aucune table n'est triée
- b : nombre de pages du tampon en mémoire centrale disponible pour le tri

Jointure par tri fusion

Trier R et S par tri externe et réécrire dans des fichiers temporaires
 Lire groupe de lignes GR(cR) de R pour la première valeur cR de clé de jointure
 Lire groupe de lignes GS(cS) de S pour la première valeur cS de clé de jointure
 TANT QUE il reste des lignes de R et S à traiter

SI cR = cS

Produire les lignes concaténées pour chacune des combinaisons de lignes de GR(cR) et GS(cS);

Lire les groupes suivants GR(cR) de R et GS(cS) de S;

SINON

SI cR < cS

Lire le groupe suivant GR(cR) de R

SINON

SI cR > cS

Lire le groupe GS(cS) suivant dans S

FINSI

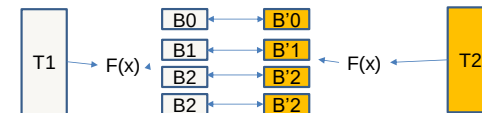
FINSI

FIN TANT QUE



Principe de la jointure par hachage

1. Hacher la plus petite des deux tables en n fragments
2. Hacher la seconde table, avec la même fonction, en n autres fragments
3. Réunir fragments par paire, et faire la jointure entre les fragments



Coût jointure par hachage

1. Chargement de T1 : $|T1|$
 2. Hachage de T1, création des groupes en RAM : 0
 3. Déchargement des groupes en MS : $|T1|$
 4. Chargement de T2 : $|T2|$
 5. Hachage de T2, création des groupes en RAM : 0
 6. Déchargement des groupes en MS : $|T2|$
 7. Chargement du Block Bi de T1 avec B'i de T2, faire la jointure entre les deux blocs : $|Bi|+|B'i|$
 8. Au total : $\sum |Bi|+|B'i|=|T1|+|T2|$
- **Coût Total = $3(|T1|+|T2|)$**



Boucle Indexée

- $T1 \bowtie_b T2$
- ❑ Comme boucles imbriquées mais avec l'utilisation de l'index défini sur l'attribut b de T2
 1. Charger une partie de T1 en RAM
 2. Charger l'index
 3. Pour chaque tuple de T1, parcourir l'index, retourner les RID des tuples de T2 à joindre
 4. Charger ces tuples en RAM
 5. Coller les tuples
- ❑ Coût = $|Index|+|T1|+|T1|*|T2|/NDIST(T2,b)$



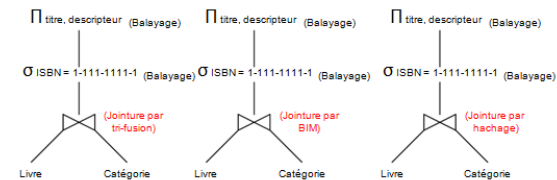
Comment se fait l'Optimisation

- ❑ Chercher le meilleur plan d'exécution?
 - coût excessif
- ❑ Solution approchée à un coût raisonnable
 - Générer les alternatives
 - heuristiques
- ❑ Choisir la meilleure estimation approximative du coût

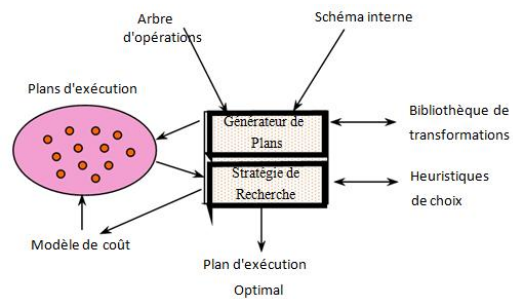


Plusieurs plans d'exécution pour un arbre algébrique

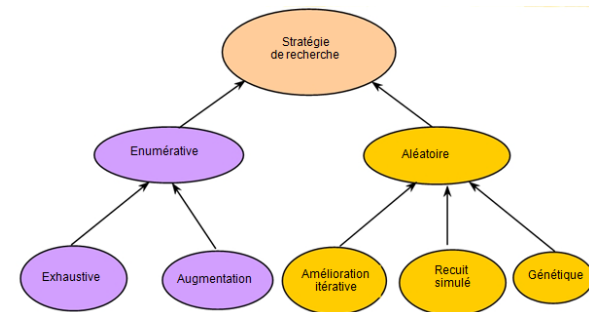
- ❑ Pour chaque opération logique
 - plusieurs choix d'opérations physiques



Choix du meilleur plan



Différentes Stratégies



Techniques d'Optimisation

❑ Vues matérialisées

- Une vue est une requête nommée
- Une vue matérialisée : les données résultant de sa requête sont stockées et maintenues

❑ Index

- Structures permettant d'associer à une clé d'un n-uplet l'adresse relative de cet n-uplet

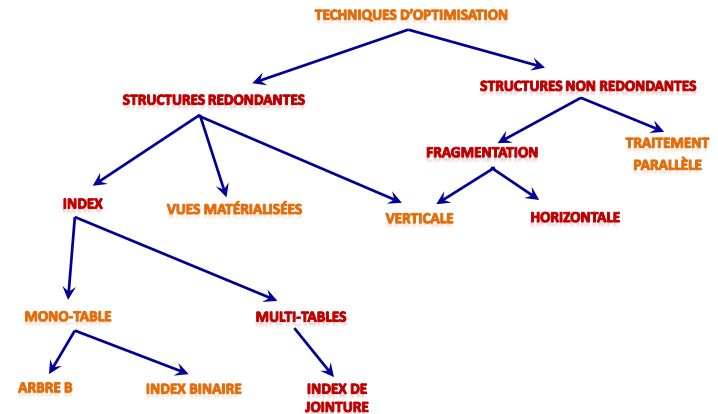
❑ Traitement parallèle

❑ Groupement

❑ Fragmentation



Classification



Maintenance des vues matérialisées

- ❑ Vues matérialisées sont calculées à partir des sources (tables)
- ❑ Mise à jour des sources implique la mise à jour des vues
- ❑ Deux méthodes de maintenance
 - Statique
 - Incrémentale



Vues matérialisées - Exemple

```
CREATE MATERIALIZED VIEW VUE1
ENABLE QUERY REWRITE
AS
SELECT *
FROM Vente, Client, Article
WHERE Vente.noClient = Client.noClient AND
Vente.noArticle = Article.noArticle
AND Client.Ville = 'Alger'
```



```
SQL> select count(*) from actvars a, timelevel t where t.tid=a.time_level and t.month_level=3;
COUNT(*)
5832000
Ecoulé : 00 :00 :06.36
Plan d'exécution
Plan hash value: 64855482
```

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		1	17	108K (2)	00:00:05
1	SORT AGGREGATE		1	17		
2	HASH JOIN		5832K	94M	108K (2)	00:00:05
3	TABLE ACCESS FULL	TIMELEVEL	2	20	3 (0)	00:00:01
4	TABLE ACCESS FULL	ACTVARS	49M	330M	108K (1)	00:00:05



```
SQL> CREATE MATERIALIZED VIEW VUE1
2  ENABLE QUERY REWRITE
3  AS
4  select count(*) from actvars a, timelevel t where t.tid=a.time_level and month_level=3 ;
Vue matérialisée créée.
Ecoulé : 00 :00 :18.64
SQL> select count(*) from actvars a, timelevel t where t.tid=a.time_level and month_level=3;
COUNT(*)
5832000
Ecoulé : 00 :00 :00.04
Plan d'exécution
Plan hash value: 2331974532
```

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		1	5	3 (0)	00:00:01
1	MAT_VIEW REWRITE ACCESS FULL	VUE1	1	5	3 (0)	00:00:01



Réécriture des requêtes

❑ Processus de réécriture:

- Après le processus de sélection des vues, toutes les requêtes définies sur l'entrepôt doivent être **réécrites** en fonction des vues

❑ Sélectionner la meilleure réécriture pour une requête est une tâche difficile.

❑ Processus supporté dans la plupart des SGBD multidimensionnels (Oracle)



Vues matérialisées

❑ Exigence de ressources:

- Espace disque
- Coût de maintenance (rafraîchissement des données)
- Coût de calcul

➡ Impossibilité de matérialiser toutes les vues

❑ Problème de Sélection des Vues (PSV):

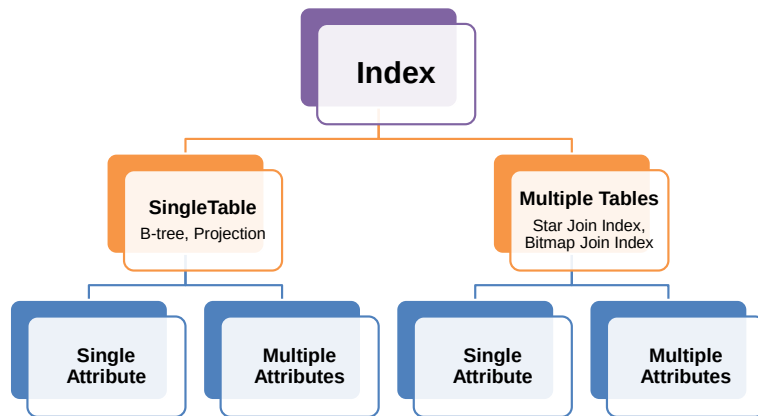
- Sélectionner un ensemble de vues afin de maximiser ou minimiser une fonction objectif sous une contrainte

- Fonction « objectif »:

Optimiser le coût d'exécution des requêtes ou Optimiser le coût de maintenance



Classification des Index



Techniques d'indexation

- ❑ Index bitmap (index binaire)
 - ❑ Populaire dans les produits OLAP
 - Microsoft SQL server, Sysbase IQ, Oracle
 - Un vecteur de bits pour chaque valeur d'attribut
- ❑ Opération bit à bit pour l'exécution de requêtes
 - Comparaison
 - Jointure
 - Agrégation
 - Plus compact que les B+ arbres (compression)



Principe de l'index bitmap

- Soit un attribut A, ayant prenant n valeurs possibles $[v_1, \dots, v_n]$ (domaine)

- Création d'un index bitmap sur l'attribut A:

- On crée n tableaux de bit, un pour chaque valeur v_i
- Le tableau contient un bit pour chaque tuple t
- Le bit d'un tuple t est à 1 si: $t.A = v_i$, à 0 sinon

```
CREATE BITMAP INDEX
Fonction_ib_idx
ON
Client.Fonction
```

Select * From Employé
where Fonction = Soudeur

Select count(*) From Employé
where Fonction IN ('Soudeur',
'Tourneur')

Select * From Employé
where Fonction IN ('Soudeur', 'Tourneur')

Table Employé				Index binaire sur l'attribut Fonction				
ROWID(RID)	N°E	Nom	Fonction	ROWID(RID)	Soudeur	Fraiseur	Sabieur	Tourneur
00055 :000 :0023	1	Karim	Fraiseur	00055 :000 :0023	0	1	0	0
00234 :020 :8922	2	Hichem	Soudeur	00234 :020 :8922	1	0	0	0
19000 :328 :6200	3	Salim	Tourneur	19000 :328 :6200	0	0	0	1
21088 :120 :1002	4	Ali	Sabieur	21088 :120 :1002	0	0	1	0
20088 :120 :1012	5	Lyes	Soudeur	20088 :120 :1012	1	0	0	0
25087 :120 :1023	6	Rabie	Tourneur	25087 :120 :1023	0	0	0	1

Soudeur	Tourneur	OR
0	0	0
1	0	1
0	1	1
0	0	0
1	0	1
0	1	1

OR

=



Index Binaire

```
CREATE BITMAP INDEX members_gender_i ON
members(gender);
```

```
EXPLAIN PLAN FOR
SELECT * FROM
members WHERE gender
= 'F';
```

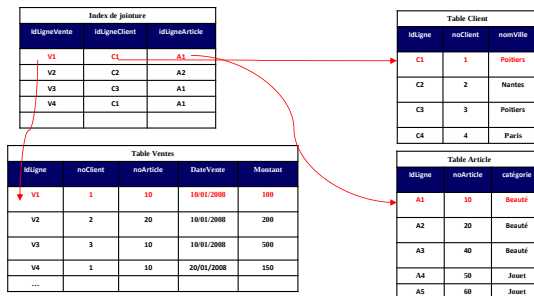
PLAN_TABLE_OUTPUT						
Plan hash value: 1733317667						
ID	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		509	128K	3 (0)	00:00:01
1	TABLE ACCESS BY INDEX ROWID BATCHED	MEMBERS	509	128K	3 (0)	00:00:01
2	BITMAP CONVERSION TO ROWIDS		1			
3	BITMAP INDEX SINGLE VALUE	MEMBERS_GENDER_I	1			
Predicate Information (identified by operation id):						
3 - access("GENDER"='M')						
Note						

- dynamic statistics used: dynamic sampling (level=2)						



Index de jointure

- Pré-calculent une opération de jointure
- Utilisés avec les tables liées avec des clés étrangères avec plusieurs tables
- Maintient des relations entre une clé étrangère (table fille) avec les clés primaires (tables mères)



Index de jointure Bitmap : Exemple

Ventes

idV	CID	PID	TID	Montant
1	616	106	11	25
2	616	106	66	28
3	616	104	33	50
4	515	104	11	10
5	414	105	66	14
6	212	106	55	14
7	111	101	44	20
8	111	101	33	27
9	212	101	11	100
10	313	102	11	200
11	414	102	11	102
12	414	102	55	103
13	515	102	66	100
14	515	103	55	17
15	212	103	44	46
16	111	105	66	44
17	212	104	66	40
18	515	104	22	20
19	111	101	33	27
20	212	101	11	100
21	313	102	11	200
22	414	102	11	102
23	414	102	55	103
24	515	102	66	100
25	515	103	55	17
26	212	103	44	46
27	111	105	66	44
28	212	104	66	40
29	515	104	22	20

Client

idC	CID	Nom	Ville
1	616	Yacine	Alger
2	616	Yacine	Alger
3	616	Yacine	Alger
4	515	Lyes	Tripoli
5	414	Yacine	Tripoli
6	212	Yacine	Alger
7	111	Rabah	Alger

Produit

idP	PID	Nom	Categori
1	106	StanoPhone	Beauté
2	104	StanoPhone	Beauté
3	105	StanoPhone	Beauté
4	102	StanoPhone	Beauté
5	103	StanoPhone	Beauté
6	101	StanoPhone	Beauté
7	101	StanoPhone	Beauté
8	101	StanoPhone	Beauté
9	101	StanoPhone	Beauté
10	102	StanoPhone	Beauté
11	102	StanoPhone	Beauté
12	102	StanoPhone	Beauté
13	102	StanoPhone	Beauté
14	103	StanoPhone	Beauté
15	103	StanoPhone	Beauté
16	103	StanoPhone	Beauté
17	103	StanoPhone	Beauté
18	103	StanoPhone	Beauté
19	103	StanoPhone	Beauté
20	103	StanoPhone	Beauté
21	103	StanoPhone	Beauté
22	103	StanoPhone	Beauté
23	103	StanoPhone	Beauté
24	103	StanoPhone	Beauté
25	103	StanoPhone	Beauté
26	103	StanoPhone	Beauté
27	103	StanoPhone	Beauté
28	103	StanoPhone	Beauté
29	103	StanoPhone	Beauté
30	103	StanoPhone	Beauté

Temps

idT	TID	Mois	Année
1	11	Jan	2019
2	66	Jan	2019
3	33	Jan	2019
4	11	Jan	2019
5	66	Jan	2019
6	33	Jan	2019
7	11	Jan	2019
8	66	Jan	2019
9	33	Jan	2019
10	11	Jan	2019
11	66	Jan	2019
12	33	Jan	2019
13	11	Jan	2019
14	66	Jan	2019
15	33	Jan	2019
16	11	Jan	2019
17	66	Jan	2019
18	33	Jan	2019
19	11	Jan	2019
20	66	Jan	2019
21	33	Jan	2019
22	11	Jan	2019
23	66	Jan	2019
24	33	Jan	2019
25	11	Jan	2019
26	66	Jan	2019
27	33	Jan	2019
28	11	Jan	2019
29	66	Jan	2019
30	33	Jan	2019

UB Ville

idV	A	B	C	D	E	F
1	1	1	0	0	0	0
2	1	1	0	0	0	0
3	1	1	0	0	0	0
4	1	1	0	0	0	0
5	1	1	0	0	0	0
6	1	1	0	0	0	0
7	1	1	0	0	0	0
8	1	1	0	0	0	0
9	1	1	0	0	0	0
10	1	1	0	0	0	0
11	1	1	0	0	0	0
12	1	1	0	0	0	0
13	1	1	0	0	0	0
14	1	1	0	0	0	0
15	1	1	0	0	0	0
16	1	1	0	0	0	0
17	1	1	0	0	0	0
18	1	1	0	0	0	0
19	1	1	0	0	0	0
20	1	1	0	0	0	0
21	1	1	0	0	0	0
22	1	1	0	0	0	0
23	1	1	0	0	0	0
24	1	1	0	0	0	0
25	1	1	0	0	0	0
26	1	1	0	0	0	0
27	1	1	0	0	0	0
28	1	1	0	0	0	0
29	1	1	0	0	0	0
30	1	1	0	0	0	0

UB Cat

idP	B	M	J	P	F
1	1	1	0	0	0
2	1	1	0	0	0
3	1	1	0	0	0
4	1	1	0	0	0
5	1	1	0	0	0
6	1	1	0	0	0
7	1	1	0	0	0
8	1	1	0	0	0
9	1	1	0	0	0
10	1	1	0	0	0
11	1	1	0	0	0
12	1	1	0	0	0
13	1	1	0	0	0
14	1	1	0	0	0
15	1	1	0	0	0
16	1	1	0	0	0
17	1	1	0	0	0
18	1	1	0	0	0
19	1	1	0	0	0
20	1	1	0	0	0
21	1	1	0	0	0
22	1	1	0	0	0
23	1	1	0	0	0
24	1	1	0	0	0
25	1	1	0	0	0
26	1	1	0	0	0
27	1	1	0	0	0
28	1	1	0	0	0
29	1	1	0	0	0
30	1	1	0	0	0

**SELECT count(*)
FROM Ventes V, Client C
WHERE V.CID = C.CID
AND C.Ville = 'Alger'**

**CREATE BITMAP INDEX
ON Ventes (Client.Ville)
WHERE Ventes.CID = Client.CID**

- Deux opérations :
1. Lire dans le bitmap la colonne P
 2. Compter le nombre de 1.
- Pas de lecture dans la table Ventes

Exemple1 : Sans Index

```
SQL> select count(*) from actvars a, timelevel t where t.tid=a.time_level and t.month_level=3;
COUNT(*)
5832000
Ecoulé : 00 :00 :06.36
Plan d'exécution
Plan hash value: 64855482
-----
| Id | Operation          | Name          | Rows  | Bytes | Cost (%CPU)| Time          |
|----|-----|-----|-----|-----|-----|-----|-----|
| 0  | SELECT STATEMENT    |               |      1 |    17 |    108k (2)| 00:00:05     |
| 1  | SORT AGGREGATE      |               |      1 |    17 |           |              |
|* 2  | HASH JOIN           |               | 5832K | 94M   |    108k (2)| 00:00:05     |
|* 3  | TABLE ACCESS FULL | TIMELEVEL     |      2 |    20 |        3 (0)| 00:00:01     |
| 4  | TABLE ACCESS FULL | ACTVARS       | 49M   | 330M  |    108k (1)| 00:00:05     |
-----
```



Exemple1 : avec index

```
SQL> create bitmap index month_idx on actvars(timelevel.month_level) from actvars, timelevel where actvars.time_level=timelevel.tid;
Index créé.
Ecoulé : 00 :00 :50.16
SQL> select count(*) from actvars a, timelevel t where t.tid=a.time_level and t.month_level=3;
COUNT(*)
5832000
Ecoulé : 00 :00 :00.27
Plan d'exécution
Plan hash value: 2154897137
-----
| Id | Operation          | Name          | Rows  | Bytes | Cost (%CPU)| Time          |
|----|-----|-----|-----|-----|-----|-----|-----|
| 0  | SELECT STATEMENT    |               |      1 |    17 |    687 (0)| 00:00:01     |
| 1  | SORT AGGREGATE      |               |      1 |    17 |           |              |
| 2  | BITMAP CONVERSION COUNT |               | 4131K | 27M   |    687 (0)| 00:00:01     |
|* 3  | BITMAP INDEX SINGLE VALUE | MONTH_IDX     |      1 |      |           |              |
-----
```



Exemple 2 : Sans Index

```
SQL> select count(*) from actvars a, timelevel t, prodlevel p where t.tid=a.time_level and p.code_level=a.product_level and month_level=3 and p.group_level='P4FR1LHPUXJ8';
```

COUNT(*)
39708
Ecoulé : 00 :00 :06.52
Plan d'exécution
Plan hash value: 984815386

SQL> select count(*) from actvars a, timelevel t, prodlevel p where t.tid=a.time_level and p.code_level=a.product_level and month_level=3 and p.group_level='P4FR1LHPUXJ8';

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		1	56	108K (2)	00:00:05
1	SORT AGGREGATE		1	56		
2	HASH JOIN		26917	1472K	108K (2)	00:00:05
3	TABLE ACCESS FULL	TIMELEVEL	2	20	3 (0)	00:00:01
4	HASH JOIN		228K	10M	108K (2)	00:00:05
5	TABLE ACCESS FULL	PRODLEVEL	30	780	33 (0)	00:00:01
6	TABLE ACCESS FULL	ACTVARS	49M	945M	108K (1)	00:00:05



Exemple 2 : Sans Index

```
SQL> select count(*) from actvars a, timelevel t, prodlevel p where t.tid=a.time_level and p.code_level=a.product_level and month_level=3 and p.group_level='P4FR1LHPUXJ8';
```

COUNT(*)
39708
Ecoulé : 00 :00 :00.03
Plan d'exécution
Plan hash value: 3564925511

SQL> select count(*) from actvars a, timelevel t, prodlevel p where t.tid=a.time_level and p.code_level=a.product_level and month_level=3 and p.group_level='P4FR1LHPUXJ8';

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		1	20	48 (0)	00:00:01
1	SORT AGGREGATE		1	20		
2	BITMAP CONVERSION COUNT		231K	4524K	48 (0)	00:00:01
3	BITMAP AND					
4	BITMAP INDEX SINGLE VALUE	GROUP_IBX				
5	BITMAP INDEX SINGLE VALUE	MONTH_IBX				



Exemple 3 : Sans Index

```
SQL> select /*+ full (actvars) */count(*) from actvars a, timelevel t, prodlevel p wh
COUNT(*)
218394
Ecoulé : 00 :00 :06.37
Plan d'exécution
Plan hash value: 984815386
```

SQL> select count(*) from actvars a, timelevel t, prodlevel p
where t.tid=a.time_level and p.code_level=a.product_level and
month_level>3 and p.group_level='P4FR1LHPUXJ8';

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		1	56	108K (2)	00:00:05
1	SORT AGGREGATE		1	56		
2	HASH JOIN		222K	11M	108K (2)	00:00:05
3	TABLE ACCESS FULL	TIMELEVEL	18	180	3 (0)	00:00:01
4	HASH JOIN		228K	10M	108K (2)	00:00:05
5	TABLE ACCESS FULL	PRODLEVEL	30	780	33 (0)	00:00:01
6	TABLE ACCESS FULL	ACTVARS	49M	945M	108K (1)	00:00:05



Exemple 3 : Avec Index

```
SQL> select count(*) from actvars a, timelevel t, prodlevel p where t.tid=a.time_level and p
COUNT(*)
218394
Ecoulé : 00 :00 :01.38
Plan d'exécution
Plan hash value: 3799251361
```

SQL> select count(*) from actvars a, timelevel t, prodlevel p
where t.tid=a.time_level and p.code_level=a.product_level and
month_level>3 and p.group_level='P4FR1LHPUXJ8';

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		1	20	76953 (2)	00:00:04
1	SORT AGGREGATE		1	20		
2	BITMAP CONVERSION COUNT		231K	4524K	76953 (2)	00:00:04
3	BITMAP AND					
4	BITMAP INDEX SINGLE VALUE	GROUP_IBX				
5	BITMAP MERGE					
6	BITMAP INDEX RANGE SCAN	MONTH_IBX				



Caractéristiques des Bitmaps

- ❑ Les opérations de comparaison, jointure et d'agrégation sont réduites à une arithmétique sur les bits, d'où un traitement plus efficace
- ❑ La réponse à une requête multidimensionnelle va consister à faire l'intersection des vecteurs de bits des diverses dimensions
- ❑ Réduction significative en espace et nombre d'accès en mémoire secondaire



Fragmentation de Données (Partitionnement)



La fragmentation de données

Définition

- ❑ Décomposer les objets de la BD (relation, index, vues) en un ensemble de morceaux appelés **Partitions**.

Fragmentation horizontale

- ❑ La table est fragmentée par rapport à ses instances en un ensemble de lignes.

Fragmentation verticale

- ❑ La table est fragmentée selon ses attributs en un ensemble de colonnes.

Fragmentation mixte

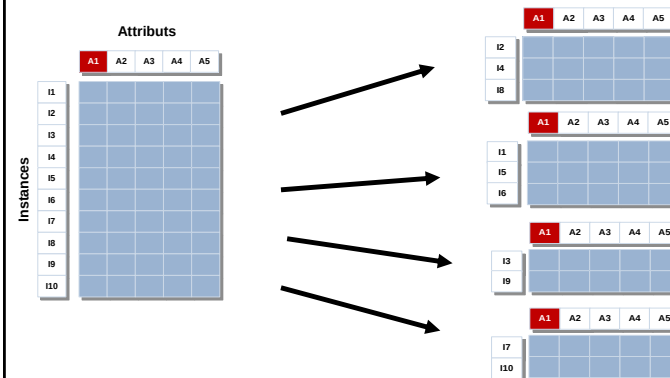
- ❑ La table est fragmentée horizontalement et verticalement.



Types de fragmentation

Fragmentation horizontale

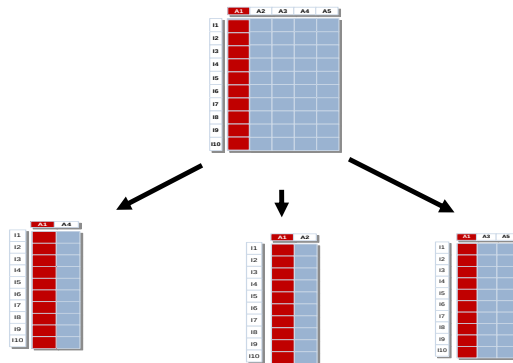
- Décomposer les objets en un ensemble de lignes (instances)



Types de fragmentation

Fragmentation verticale

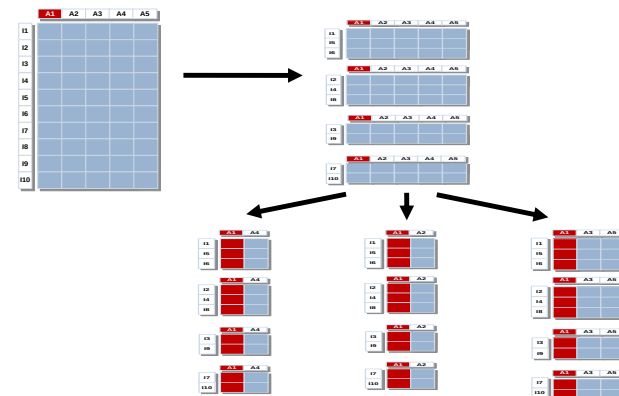
- Décomposer les objets en un ensemble de colonnes (attributs).



Types de fragmentation

Fragmentation mixte

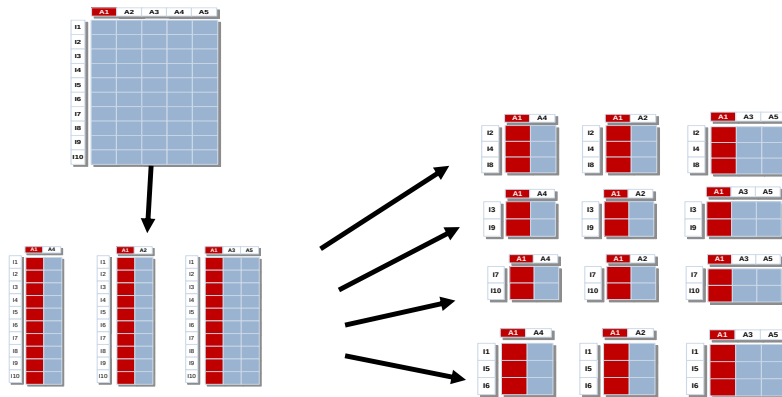
- ☐ Horizontale suivie d'une verticale.



Types de fragmentation

Fragmentation mixte

- ☐ Verticale suivie d'une horizontale



Fragmentation Horizontale Primaire et dérivée (I)

Fragmentation horizontale primaire (FHP)

- Fragmenter une table en utilisant les prédicats de sélection définis sur cette table

Prédicat : Attribut θ Valeur, $\theta \in \{=, <, \leq, >, \geq\}$ et valeur $\in \text{Domaine}(\text{Attribut})$.

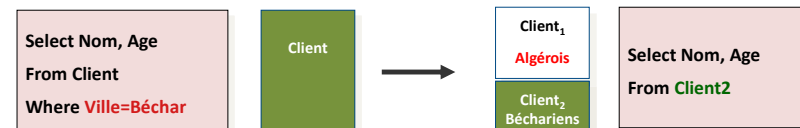
- Exemple: Client (Client_id, Nom, Ville)

- Client₁ : $\sigma_{\text{Ville}='Alger'}(\text{Client})$

- Client₂ : $\sigma_{\text{Ville}='Béchar'}(\text{Client})$

Impact de la FHP sur les requêtes

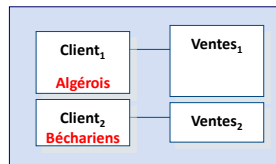
- Optimisation des sélections



Fragmentation Horizontale Primaire et dérivée (II)

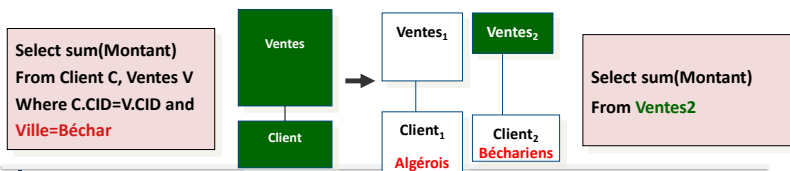
Fragmentation horizontale dérivée (FHD)

- Fragmenter une table (S) selon des attributs d'une autre table (T) : (existence de lien entre S et T)
- Ventes(Client_id, Produit_id, Date, Montant)
 - $Ventes_1 = Ventes \times Client_1$
 - $Ventes_2 = Ventes \times Client_2$



Impact de la FHD sur les requêtes

- Optimisation de la jointure entre S et T

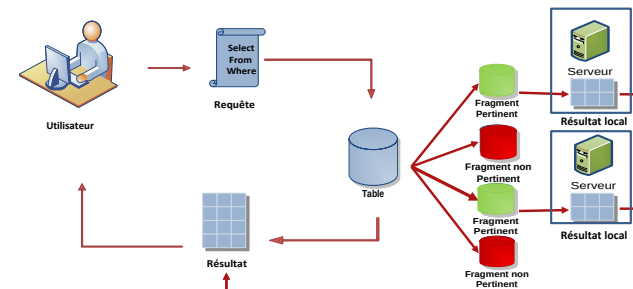


Intérêt de la fragmentation horizontale(I)

Améliorer la performance

1. Partition Elimination (Pruning)

- Elimination des partitions non pertinentes
- Possibilité d'exécution parallèle des sous requêtes

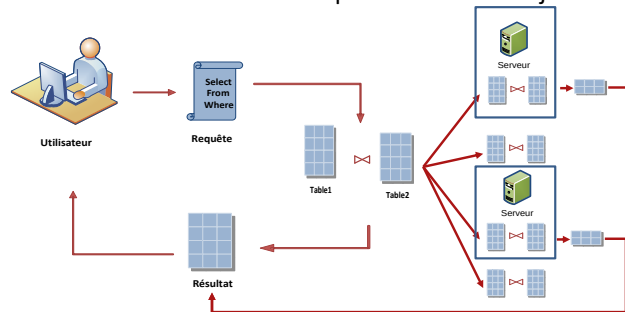


Intérêt de la fragmentation horizontale(II)

Amélioration de la performance

2. Partition Wise Joins

- ☐ Elimination des jointures non pertinentes
- ☐ Possibilité d'exécution parallèle des sous-jointures



Intérêt de la fragmentation horizontale(III)

Améliorer la manageabilité

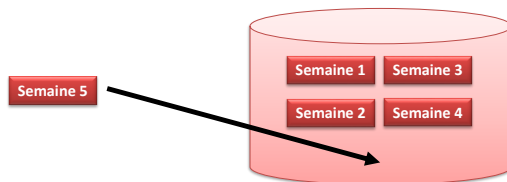
- Fragmentation horizontale préserve le schéma logique
 - Toutes les opérations se font au niveau partition
 - Possibilité de manipulation individuelle ou collective des partitions
- Manipuler une partition à la fois.
 - Possibilité de définir des index locaux aux partitions
 - Existences de plusieurs Fonctions de manipulation des partitions

Fonction	Signification
ADD PARTITION	Ajouter une partition à une table déjà fragmentée
COLESCCE PARTITION	Redistribuer les tuples d'une partition dans les autres partitions
DROP PARTITION	Supprimer une partition ainsi que son contenu
EXCHANGE PARTITION	Convertir une table non fragmentée en une partition d'une autre table et inversement
MERGE PARTITION	Fusionner deux partitions dans une seule
SPLIT PARTITION	Eclater une partition en deux partitions
TRUNCATE PARTITION	Vider une partition sans la supprimer

Exemple d'ajout de données

Alimentation hebdomadaire d'une table

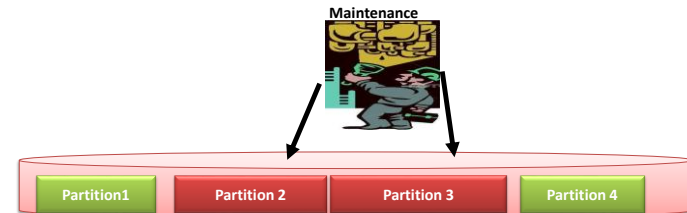
- L'administrateur partitionne sa table par semaine
- L'alimentation de l'entrepôt consiste alors à ajouter une partition à la table
- Aucune autre partition ne sera touchée.



Intérêt de la fragmentation horizontale(IV)

Améliorer la maintenance et la disponibilité

- La manipulation au niveau partition permet
 - Cibler la maintenance sur certaines partitions
 - Minimiser le temps de maintenance
- La possibilité de stocker les partitions sur des emplacements différents permet
 - L'effet des pannes de disques est limité
 - Partitions saines toujours disponibles

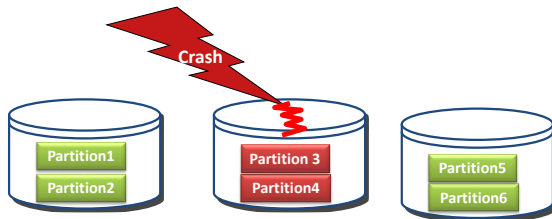


Améliorer la maintenance et la disponibilité

Améliorer la maintenance et la disponibilité

- La possibilité de stocker les partitions sur des emplacements différents permet

- L'effet des pannes de disques est limité
- Partitions saines toujours disponibles



Fragmentation horizontale et index

- Un index peut être défini sur une table fragmentée ou non
- Un index défini sur une table fragmentée peut être fragmenté ou non.

Index global

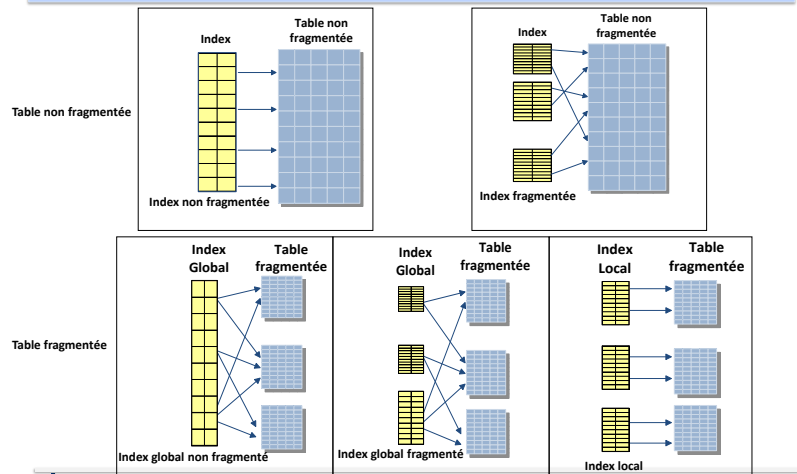
- Peut être non fragmenté défini sur une table fragmentée
- Peut être fragmenté où chaque partition de l'index référence plusieurs partitions de la table

Index local

- Equi-partitionné avec la table qu'il référence
- Chaque partition de l'index référence une et une seule partition de la table



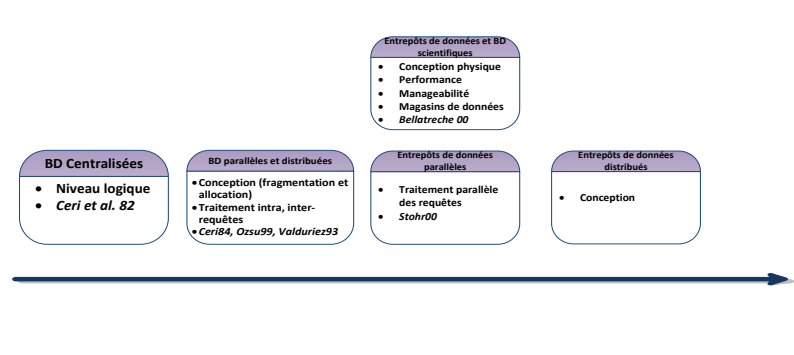
Fragmentation et index



Evolution de la fragmentation horizontale

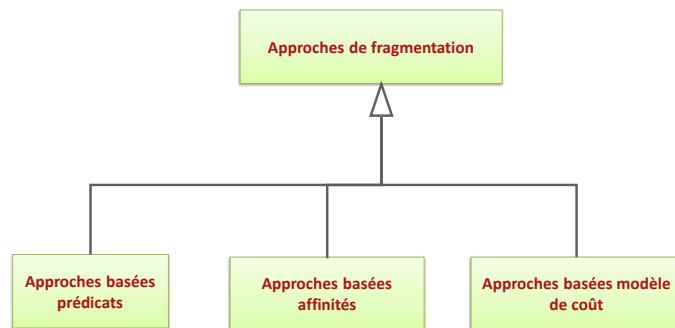


Evolution Académique



Evolution Académique (I)

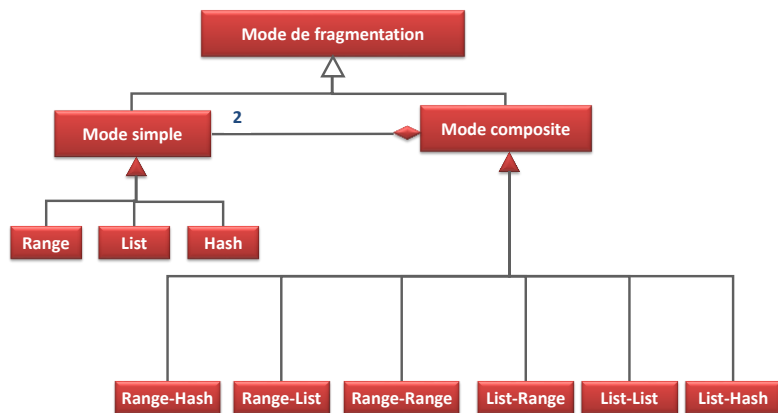
Approches de fragmentation



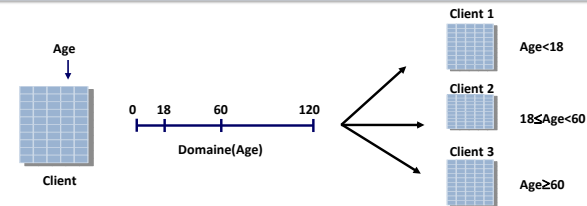
Evolution industrielle



Modes de fragmentation



Fragmentation par intervalle « Range »

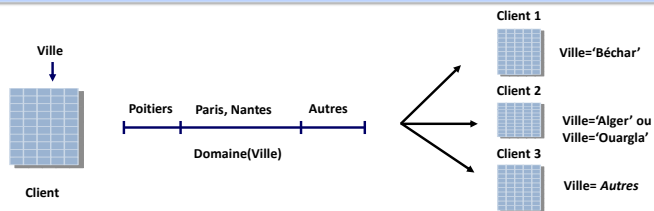


```

CREATE TABLE Client
(Client_id NUMBER(5),
Nom Varchar2(20),
Ville Varchar2(20),
Age Number(3),
Genre Varchar2(1))
PARTITION BY RANGE (Age)
(
PARTITION Client_Moins_18 VALUES LESS THAN (18) TABLESPACE TBSPMoins27,
PARTITION Client_18_59 VALUES LESS THAN (60) TABLESPACE TBSP27-59,
PARTITION Client_60_Et_Plus VALUES LESS THAN (MAXVALUES) TABLESPACE TBSPPlus60
);
  
```



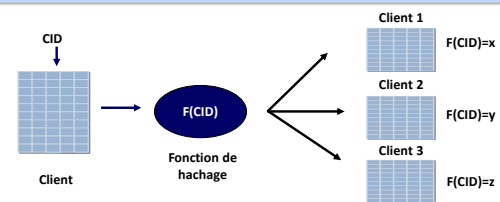
Fragmentation par liste « List »



```
CREATE TABLE Client
(Client_id NUMBER(5),
Nom Varchar2(20),
Ville Varchar2(20),
Age Number(3),
Genre Varchar2(1))
PARTITION BY List (Ville)
(
PARTITION Client_Béchar VALUES ('Béchar') TABLESPACE TBSBECHAR,
PARTITION Client_Alg_Oua VALUES ('Alger','Ouargla') TABLESPACE TBSALGOUA,
PARTITION Client_Autres VALUES (DEFAULT) TABLESPACE TBSAUTRES
);
```



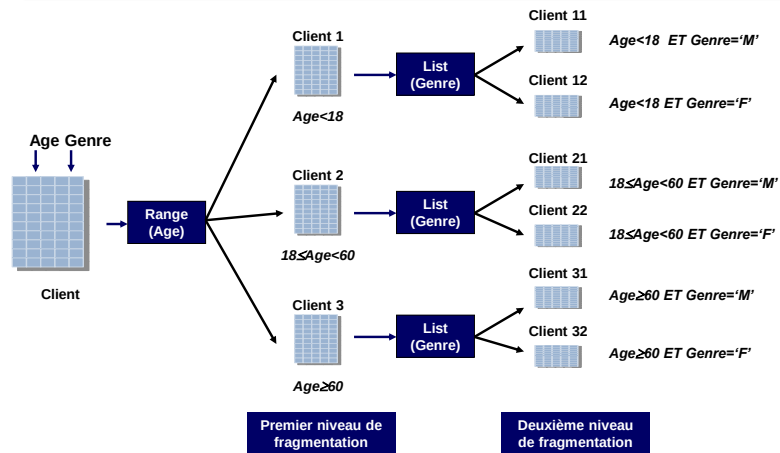
Fragmentation par hachage « Hash »



```
CREATE TABLE Client
(Client_id NUMBER(5),
Nom Varchar2(20),
Ville Varchar2(20),
Age Number(3),
Genre Varchar2(1))
PARTITION BY Hash (CID)
(
PARTITIONS 3
STORE IN (TBS1, TBS2, TBS3)
);
```



Modes de fragmentation composite

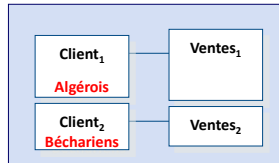


Modes de fragmentation composite

```
CREATE TABLE Client
(Client_id NUMBER(5),
Nom Varchar2(20),
Ville Varchar2(20),
Age Number(3),
Genre Varchar2(1))
PARTITION BY RANGE (Age)
SUBPARTITION BY LIST (Genre)
SUBPARTITION TEMPLATE(
SUBPARTITION Client1 VALUES ('M') TABLESPACE TBSMasculin ,
SUBPARTITION Client2 VALUES ('F') TABLESPACE TBSFéminin )
(
PARTITION Client_Moins_18 VALUES LESS THAN (18),
PARTITION Client_18_59 VALUES LESS THAN (60),
PARTITION Client_60_Et_Plus VALUES LESS THAN (MAXVALUES)
);
```

Fragmentation dérivée par le mode Référence

- Fragmenter une table selon le schéma de fragmentation d'une autre table en utilisant le lien par clé étrangère.

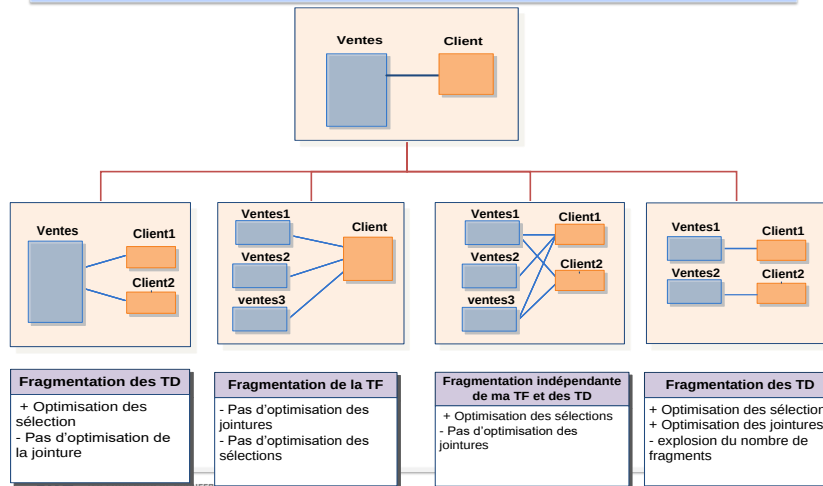


```
CREATE TABLE Ventes
(Client_id NUMBER(5),
Time_id NUMBER(5),
Montant NUMBER(20),
CONSTRAINT order_items_fk
FOREIGN KEY(Client_id) REFERENCES Client(Client_id)
)
PARTITION BY REFERENCE(order_items_fk);
```



Sélection d'un schéma de fragmentation
horizontale pour un entrepôt de
données relationnel

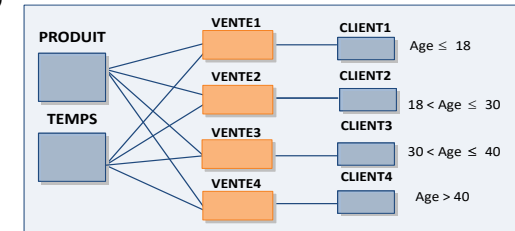
Scénarii de fragmentation d'un ED relationnel



Notre démarche de fragmentation

1. Fragmenter (virtuellement/physiquement) des tables de dimension en utilisant la fragmentation primaire
2. Fragmenter la tables des faits (en utilisant les schémas de fragmentation des tables de dimension)

Exemple



Explosion du nombre de fragments

$$N = \prod_{i=1}^k M_i$$

M_i : nombre de fragments de la table de dimension D_i
 k : nombre de tables de dimension fragmentées